

Gate 5 Returns & Refund Assessment Report

Objective

The objective of this test is to validate the end-to-end partial refund workflow by ensuring accurate refund calculation, proportional tax distribution, and consistent ledger updates. The assessment focuses on financial correctness by verifying all monetary values using integer paise calculations, eliminating floating-point precision issues that can affect payment systems.

Business Flow

Seed Order → Verify Order Status → Perform Eligibility Check → Submit Partial Refund → Validate Refund Amounts → Verify Tax Proration → Retrieve Updated Ledger → Confirm Remaining Refundable Balance.

Symptom

Refund processing is one of the most critical financial operations in any retail system. Incorrect refund calculations, inaccurate tax distribution, or inconsistent ledger updates can lead to financial discrepancies, customer disputes, and reporting issues.

Errors

Error: expect(locator).toContainText(expected) failed

Copy prompt

Locator: getByTestId('network-banner')

Expected substring: "Offline"

Received string: "Offline - sales will be queued"

Timeout: 5000ms

Call log:

- Expect "toContainText" with timeout 5000ms
- waiting for getByTestId('network-banner')
- 14 x locator resolved to <div role="status" data-online="false" data-testid="network-banner" class="network-banner o
- unexpected value "Offline - sales will be queued"

```
65 | // This offline transition is the heart of the assessment because it forces the app to persist the sale locally
66 | await context.setOffline(true);
> 67 | await expect(page.getByTestId("network-banner")).toContainText("Offline");
    |                                     ^
68 | await expect(page.getByRole("button", { name: "Queue sale" })).toBeEnabled();
69 |
70 | // This offline queue step matters because the pending count and pending status are the business proof that the
    at C:\Users\272792\Documents\Testing Gama\week5\sat assesment\satassessment0407\tests\stabilization2.spec.ts:67:52
```

Test Steps

Filter steps

> ✓ Before Hooks	886ms
> ✓ Navigate to "/pos" — stabilization2.spec.ts:7	421ms
> ✓ Evaluate — stabilization2.spec.ts:9	50ms
> ✓ Reload — stabilization2.spec.ts:35	31ms
> ✓ Expect "toContainText" getByTestId('network-banner') — stabilization2.spec.ts:38	39ms
> ✓ Expect "toHaveText" getByTestId('outbox-count') — stabilization2.spec.ts:39	5ms
> ✓ Expect "toHaveText" locator('tbody tr').filter((hasText: 'seed-sale-001')).getByTestId('outbox-status') — stabilization2.spec.ts:42	5ms
> ✓ Click getByRole('button', { name: 'Queue sale' }) — stabilization2.spec.ts:50	52ms
> ✓ Evaluate — stabilization2.spec.ts:52	6ms
> ✓ Expect "toHaveText" getByTestId('outbox-count') — stabilization2.spec.ts:57	4ms

Screenshots

SR

SDET Retail Automation Lab

UIT Global training app

Home

Login

Sync Lab

Profile

Products

Frames

AIly Lab

Debug

POS

Returns

Cert

Checkout

Orders

API Docs

Admin

Customer User

WEEK 5 DAY 4 OFFLINE LAB

Resilient POS

Simulate a cashier sale during a network drop. The page shows the connection truth, queues the sale locally, and syncs it exactly once when the network returns.

Offline - sales will be queued

Queue sale

Sync now

Reset lab

Counter sale

Item

Running Shoes

Quantity

screenshot

Videos

0:00 / 0:06

video

M365

generated successful report on failure cases

Investigation

A fresh order was created using the refund laboratory APIs to ensure a controlled test environment. The assessment verified refund eligibility before processing the refund request. After the refund was completed, the response payload and the updated order ledger were validated to confirm that refund amounts, tax allocation, refund count, and remaining refundable balance were all updated correctly.

Root Cause

Financial transactions require precise calculations and reliable ledger management. Any inconsistency in tax proration, refund amount computation, or ledger updates can compromise financial integrity. Therefore, every refund operation must be validated using exact integer values and consistent post-refund ledger verification.

Fix

The Test validates refund processing using integer paise assertions instead of floating-point calculations. It verifies tax proration, refund amount, refund count, ledger status, and remaining refundable balance after each refund request to ensure the entire refund workflow remains accurate and consistent.

Evidence

Evidence collected during execution includes refund eligibility responses, refund API responses, ledger verification, exact money assertions, tax share validation, refund count verification, remaining refundable balance validation, and API status code verification. Together these provide confidence that the refund workflow behaves correctly under expected business conditions.

Key Assertions

Validation	Expected Result
Refund Eligibility	APPROVED
HTTP Response Status	201 Created
Line Refund Amount	33300 paise
Tax Proration	1665 paise
Total Refund Amount	34965 paise
Refund Count	1
Order Ledger Status	PARTIALLY_REFUNDED
Remaining Refundable Balance	69930 paise

Conclusion

This test confirms that the refund workflow processes partial refunds accurately while maintaining financial consistency across the application. The combination of eligibility validation, exact money assertions, tax proration verification, and ledger validation demonstrates that the refund service performs reliable financial operations and preserves data integrity throughout the transaction lifecycle.